

CSE 713 — Artificial Intelligence

Game Playing: Model Answers (2020–2024)

University of Chittagong · 7th Semester B.Sc. Engineering

Topics: Minimax Algorithm · Alpha-Beta Pruning · Cutoffs · Quiescence

Contents

Quick Reference	2
1 Standard Tree Used in 2023 and 2024	2
2 2020 — Q3d (2.5 marks): Minimax Algorithm	3
3 2021 — Q3d (4 marks): Alpha-Beta on Small Tree	3
4 2022 — A-4b,c,d (7 marks total)	4
4.1 A-4b (4 marks): Minimax Search Procedure	4
4.2 A-4c (2 marks): α -cutoff, β -cutoff, Futility-cutoff	5
4.3 A-4d (1 mark): Waiting for Quiescence	6
5 2023 — Q3b (4 marks): Alpha-Beta on Standard Tree	6
6 2024 — Q3a (5 marks): Full Alpha-Beta Question	7
Exam Cheat-Sheet	8

Quick Reference: All Chapter 5 Past-Paper Questions

Year	Q#	Topic	Marks	Section
2024	Q3a	Alpha-Beta pruning (full: limitations + trace + analysis)	5	A
2023	Q3b	Alpha-Beta pruning (same tree, illustrate time-complexity benefit)	4	A
2022	A-4b	Describe minimax search procedure	4	A
2022	A-4c	Explain α -cutoff, β -cutoff, Futility-cutoff with figures	2	A
2022	A-4d	What is “Waiting for Quiescence”?	1	A
2021	Q3d	Alpha-Beta pruning on given tree	4	A
2020	Q3d	Describe Minimax algorithm for game trees	2.5	A

Pattern: Chapter 5 questions appear *every single year* in Section A Q3. The 2023 and 2024 papers use the **identical game tree** — memorise the trace cold. **Key result:** Root A = **9**. Branches I and J (under C) are pruned.

Standard Tree Used in 2023 and 2024

Both 2023 Q3b and 2024 Q3a use the same tree. Memorise this structure.

Level	Tree structure
-----	-----
	<pre> A [MAX] / \ B C D [MIN nodes] / \ / \ E F G H I J K [MAX nodes] /\ /\ /\ /\ /\ /\ /\ LM NO PQ RS TU VW XY [leaves] 75 4(-5) 23 0(-2) 62 58 90 </pre>
Leaf values: L=7, M=5, N=4, O=-5, P=2, Q=3, R=0, S=-2, T=6, U=2, V=5, W=8, X=9, Y=0	

Bottom-up Minimax values (no pruning):

$$E = \max(7, 5) = 7 \quad F = \max(4, -5) = 4 \quad G = \max(2, 3) = 3$$

$$B = \min(E, F, G) = \min(7, 4, 3) = \mathbf{3}$$

$$H = \max(0, -2) = 0 \quad I = \max(6, 2) = 6 \quad J = \max(5, 8) = 8$$

$$C = \min(H, I, J) = \min(0, 6, 8) = \mathbf{0}$$

$$K = \max(9, 0) = 9 \quad D = \min(K) = \mathbf{9}$$

$$A = \max(B, C, D) = \max(3, 0, 9) = \mathbf{9}$$

2020 — Q3d (2.5 marks): Minimax Algorithm

Question: Describe the Minimax algorithm for searching game trees.

Minimax Algorithm

Minimax determines the **optimal move** for the MAX player in a two-player, zero-sum game with perfect information, assuming the opponent (MIN) plays optimally.

Setup:

- The game tree alternates between MAX nodes (your turn) and MIN nodes (opponent's turn).
- The **root** is a MAX node. Each level alternates MAX/MIN down to a fixed depth (ply).
- **Terminal nodes** are assigned values by the **static evaluation function** $f(n)$: $f > 0$ favours MAX; $f < 0$ favours MIN.

Algorithm (depth-first, bottom-up):

1. Create the root as a MAX node with the current board configuration.
2. Expand all nodes down to the search depth (ply) using depth-first search.
3. Apply the evaluation function $f(n)$ to every **leaf** node.
4. **Back up values** toward the root:
 - At a **MAX node**: backed-up value = *maximum* of its children's values.
 - At a **MIN node**: backed-up value = *minimum* of its children's values.
5. At the root (MAX node), **choose the child whose backed-up value equals the root's value**. That child's move is the optimal move.

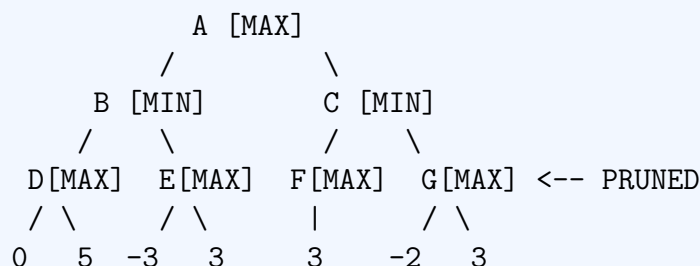
Complexity: Time $O(b^d)$; Space $O(bd)$ (linear, depth-first). Here b = branching factor, d = depth.

Key assumption: MAX always assumes MIN will take the move that *minimises* MAX's payoff (worst-case reasoning).

2021 — Q3d (4 marks): Alpha-Beta on Small Tree

Question: Explain how Alpha-Beta pruning improves game playing in the following example.

Tree structure:



Step 1: Minimax values (without pruning)

$$\begin{aligned}
D &= \max(0, 5) = 5, & E &= \max(-3, 3) = 3, & B &= \min(5, 3) = \mathbf{3} \\
F &= \max(3) = 3, & G &= \max(-2, 3) = 3, & C &= \min(3, 3) = \mathbf{3} \\
A &= \max(B, C) = \max(3, 3) = \mathbf{3}
\end{aligned}$$

Step 2: Alpha-Beta trace

Node	Type	α (init $\rightarrow$ final)	β (init $\rightarrow$ final)	Returns
A	MAX	$-\infty \rightarrow 3$	$+\infty$	3
B	MIN	$-\infty$	$+\infty \rightarrow 5 \rightarrow 3$	3
D	MAX	$-\infty \rightarrow 5$	$+\infty$	5
E	MAX	$-\infty \rightarrow 3$	5	3
C	MIN	$\alpha=3$ (from A)	$+\infty \rightarrow 3 \Rightarrow$ PRUNE	3
F	MAX	3	$+\infty$	3
G	MAX	[G PRUNED]		—

Step 3: Explanation of pruning

After A processes B, it has $\alpha_A = 3$ (MAX is guaranteed at least 3).

When C processes F and gets value 3, C's $\beta = 3$.

Condition: $\beta_C (= 3) \leq \alpha_A (= 3)$. Pruning occurs (beta-cutoff).

G is pruned because: *regardless of what G returns, MIN (C) can guarantee at most 3, which cannot improve on A's existing guarantee of 3.*

How Alpha-Beta improves game playing: It achieves the *exact same result* as Minimax (root value = 3) while skipping node G and its subtree entirely. In the best case, Alpha-Beta reduces time from $O(b^d)$ to $O(b^{d/2})$, effectively doubling the search depth within the same time budget.

2022 — A-4b,c,d (7 marks total)**A-4b (4 marks): Minimax Search Procedure**

Question: Describe the minimax search procedure.

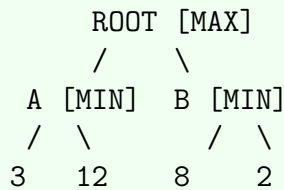
The Minimax search procedure finds the best move for MAX in a two-player zero-sum game.

Procedure:

1. **Build the game tree** starting from the current board position (root = MAX node), expanding all legal moves to a specified depth (ply).
2. **Evaluate leaf nodes** using a static *evaluation function* $f(n)$: $f(n) \gg 0$ = win for MAX; $f(n) \ll 0$ = win for MIN; $f(n) \approx 0$ = neutral.
3. **Back up values** (depth-first, bottom-up):

- **MAX node** $\leftarrow$ *maximum* of children's backed-up values.
 - **MIN node** $\leftarrow$ *minimum* of children's backed-up values.
4. **Select move:** at the root, choose the child with the value that equals the root's backed-up value.

Example (3-node tree, depth 2):



$$A = \min(3, 12) = 3$$

$$B = \min(8, 2) = 2$$

$$\text{ROOT} = \max(3, 2) = 3 \Rightarrow \text{choose branch A}$$

Complexity: Time $O(b^d)$; Space $O(bd)$.

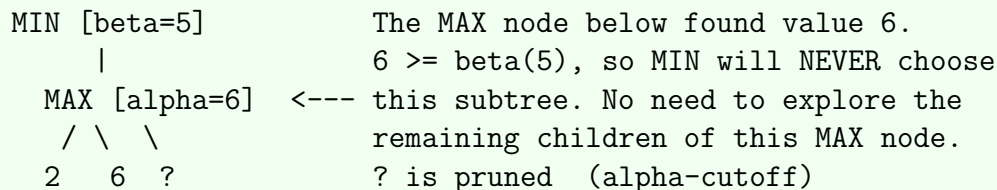
Key difference from state-space search: Minimax searches for *one best move*, not a path to a goal. No arc costs — only the evaluation function matters.

A-4c (2 marks): α -cutoff, β -cutoff, Futility-cutoff

Question: Explain with figures: (i) α -cutoff (ii) β -cutoff (iii) Futility-cutoff.

(i) **α -cutoff** (prune at a MAX node)

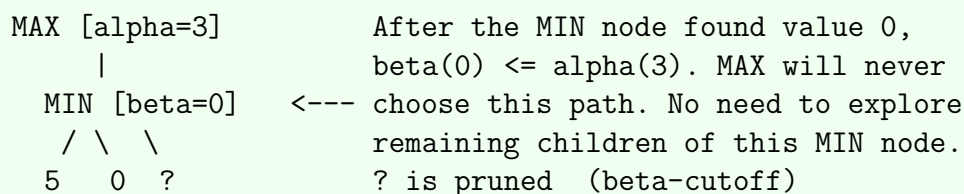
Occurs when a MAX node's α value $\geq$ its ancestor MIN node's β .



Rule: At MAX node n : if $\alpha(n) \geq \beta(n)$, prune remaining children.

(ii) **β -cutoff** (prune at a MIN node)

Occurs when a MIN node's β value $\leq$ its ancestor MAX node's α .



Rule: At MIN node n : if $\beta(n) \leq \alpha(n)$, prune remaining children.

(iii) **Futility-cutoff**

A *forward pruning* technique. Even before evaluating children, if the static evaluation $f(n)$ of a node is already so far below the current α that even the maximum possible improvement (the “futility margin” M) cannot exceed α , prune the entire subtree.

If $f(n) + M \leq \alpha \Rightarrow \text{prune (futility cutoff)}$

- M = maximum gain achievable from any single move (domain-specific constant).
- Avoids wasting time on positions that are hopelessly bad.
- **Not guaranteed safe** (may occasionally prune a good move) — used near the leaf level.

A-4d (1 mark): Waiting for Quiescence

Question: What do you mean by “Waiting for Quiescence”?

Quiescence search addresses the *horizon effect*: at the fixed search depth, applying the static evaluator to a “noisy” position (one with ongoing captures or checks) gives a misleading score.

Solution: When the depth limit is reached, instead of immediately calling $f(n)$, *continue searching* until the position becomes **quiescent** — meaning no captures, checks, or other forcing moves remain. Only then apply the evaluator.

- Prevents the case where a piece about to be captured looks “safe” at the horizon.
- Used in practice by nearly all strong game-playing programs (e.g., Deep Blue).
- The quiescence search is typically much shallower than the main search.

2023 — Q3b (4 marks): Alpha-Beta on Standard Tree

Question: By analysing the game tree ($A \rightarrow B, C, D \rightarrow E-K \rightarrow 14$ leaves), illustrate how alpha-beta pruning minimises the time complexity of Min-Max by applying the concept of pruning.

Tree and minimax values: See Section 1 (page 2). Root $A = 9$.

Alpha-Beta trace (abbreviated for 4-mark answer):

Processing B (MIN), $\alpha = -\infty$, $\beta = +\infty$:

- E (MAX): sees $L=7$, $M=5 \Rightarrow \alpha_E=7$, returns **7**. B: $\beta_B=7$.
- F (MAX): sees $N=4$, $O=-5 \Rightarrow \alpha_F=4$, returns **4**. B: $\beta_B=4$.
- G (MAX): sees $P=2$, $Q=3 \Rightarrow \alpha_G=3$, returns **3**. B: $\beta_B=3$.
- B returns **3**. A: $\alpha_A=3$.

Processing C (MIN), $\alpha=3$ (from A), $\beta=+\infty$:

- H (MAX): sees $R=0$, $S=-2 \Rightarrow$ returns **0**. C: $\beta_C=0$.
- **Check:** $\beta_C = 0 \leq \alpha_A = 3 \Rightarrow$ **BETA-CUTOFF at C!**

- **[I (leaves T=6, U=2) PRUNED] and [J (leaves V=5, W=8) PRUNED].**

Processing D (MIN), $\alpha=3$, $\beta=+\infty$:

- K (MAX): sees $X=9$, $Y=0 \Rightarrow$ returns **9**. D: $\beta_D=9$. D returns **9**.
- A: $\alpha_A=9$. **Root A = 9.**

Result: 4 leaf nodes (T, U, V, W) and 2 internal MAX nodes (I, J) were never visited.

Without pruning: 14 leaves evaluated. With Alpha-Beta: only **8 leaves** evaluated.

This demonstrates the $O(b^{d/2})$ best-case improvement over Minimax's $O(b^d)$.

Why does pruning work here? After seeing B=3, MAX (A) is guaranteed at least 3. When C's first child H returns 0, MIN (C) can achieve at most 0 from this subtree. Since $0 < 3$, MAX will *never* choose C's subtree regardless of what I and J return. Exploring them wastes time without changing the outcome.

2024 — Q3a (5 marks): Full Alpha-Beta Question

Question: Consider the game tree ($A \rightarrow B, C, D \rightarrow E-K \rightarrow 14$ leaves with values 7,5,4,-5,2,3,0,-2,6,2,5,8,9, All static scores from first player's (MAX) point of view.

Part (i): Two limitations of pure Minimax (1 mark)

1. **Exponential time complexity $O(b^d)$:** With branching factor b and depth d , the number of nodes grows exponentially. Chess has $b \approx 35$ and $d \approx 100$, giving 35^{100} nodes — computationally infeasible. Even for modest depth, the tree is too large to evaluate fully in real time.
2. **Evaluates irrelevant nodes:** Minimax explores branches that *cannot possibly* influence the final decision. Once a better option is found, the remaining subtrees under a particular parent are irrelevant — yet Minimax evaluates them anyway. This wastes computational resources that could be used for deeper search.

Part (ii): Full Alpha-Beta trace (3 marks)

Tree and minimax values: See Section 1. Root A = 9.

α/β at every node as updated:

Node	Type	Inherits	α updates	β updates
A	MAX	$-\infty, +\infty$	$-\infty \rightarrow 3 \rightarrow 9$	$+\infty$
B	MIN	$-\infty, +\infty$	$-\infty$	$+\infty \rightarrow 7 \rightarrow 4 \rightarrow 3$
E	MAX	$-\infty, +\infty$	$-\infty \rightarrow 7$	$+\infty \Rightarrow$ returns 7
F	MAX	$-\infty, 7$	$-\infty \rightarrow 4$	7 $\Rightarrow$ returns 4
G	MAX	$-\infty, 4$	$-\infty \rightarrow 3$	4 $\Rightarrow$ returns 3
C	MIN	3, $+\infty$	3 unchanged	$+\infty \rightarrow 0$ PRUNE
H	MAX	3, $+\infty$	3 ($0 < \alpha$, no update)	$+\infty \Rightarrow$ returns 0
[I PRUNED]	MAX		<i>Never visited — beta-cutoff at C</i>	
[J PRUNED]	MAX		<i>Never visited — beta-cutoff at C</i>	
D	MIN	3, $+\infty$	3 unchanged	$+\infty \rightarrow 9$
K	MAX	3, $+\infty$	3 $\rightarrow 9$	$+\infty \Rightarrow$ returns 9

Branches pruned: I (with leaves T=6, U=2) and J (with leaves V=5, W=8) under C.

Pruning reason: After H returns 0, MIN node C has $\beta_C = 0$. Since $\beta_C (= 0) \leq \alpha_A (= 3)$: MAX already has a guaranteed value of 3 from B. C can deliver *at most* 0 to A, so A will never choose the C subtree. Evaluating I and J cannot change A's final decision.

Final Minimax value returned to root: $A = 9$

Part (iii): How Alpha-Beta addresses Minimax limitations (1 mark)

- Time complexity:** Alpha-Beta reduces time from $O(b^d)$ to $O(b^{d/2})$ in the best case. This means with the same time budget, Alpha-Beta can search *twice as deep* as Minimax — or evaluate far fewer nodes at the same depth. In the above example, 6 nodes (I, J, T, U, V, W) were eliminated.
- Unnecessary node expansion:** Alpha-Beta proves certain branches are irrelevant *before* expanding them. Once $\beta_C \leq \alpha_A$, the entire $C \rightarrow I$ and $C \rightarrow J$ subtrees are provably unable to change the root decision. They are skipped without loss of accuracy.
- Decision-making under resource limits:** Because Alpha-Beta effectively halves the required search depth, a system operating under time constraints can search deeper and *consider more future consequences* within the same time window. This leads to stronger, more informed decisions in real game play.

Exam Cheat-Sheet: Key Numbers to Memorise

Minimax complexity:	$O(b^d)$ time, $O(bd)$ space (depth-first)
Alpha-Beta best case:	$O(b^{d/2})$ — search twice as deep
Alpha-Beta worst case:	$O(b^d)$ — no pruning (very poor move ordering)
2023/2024 tree result:	Root A = 9. Pruned: I(6,2), J(5,8) under C.
2021 tree result:	Root A = 3. Pruned: G(-2,3) under C.
Prune condition (both cutoffs):	$\alpha \geq \beta$
α lives at:	MAX nodes (starts $-\infty$, only increases)
β lives at:	MIN nodes (starts $+\infty$, only decreases)
Beta-cutoff:	At MIN node, $\beta \leq \alpha_{\text{ancestor}} \Rightarrow$ prune children
Alpha-cutoff:	At MAX node, $\alpha \geq \beta_{\text{ancestor}} \Rightarrow$ prune children
Futility-cutoff:	$f(n) + \text{margin} \leq \alpha \Rightarrow$ prune (forward pruning)
Quiescence:	Continue search past depth until no captures/checks remain